Ignition Perspective CSS Animation

This resource provides a detailed guide on how to perform basic animations in the Ignition Perspective module using CSS. The guide covers how to create smooth motion, pause and resume animations, and synchronize two robots using a state machine. The project starts with basic animations and then demonstrates how to create more complex movements. Three UDTs were used to ensure that multiple robots animate in the proper sequence.

Table of Contents

Basic....2
- Properties....2
  - transformOrigin....2
  - transform....2
  - transition....2
- In Designer....3

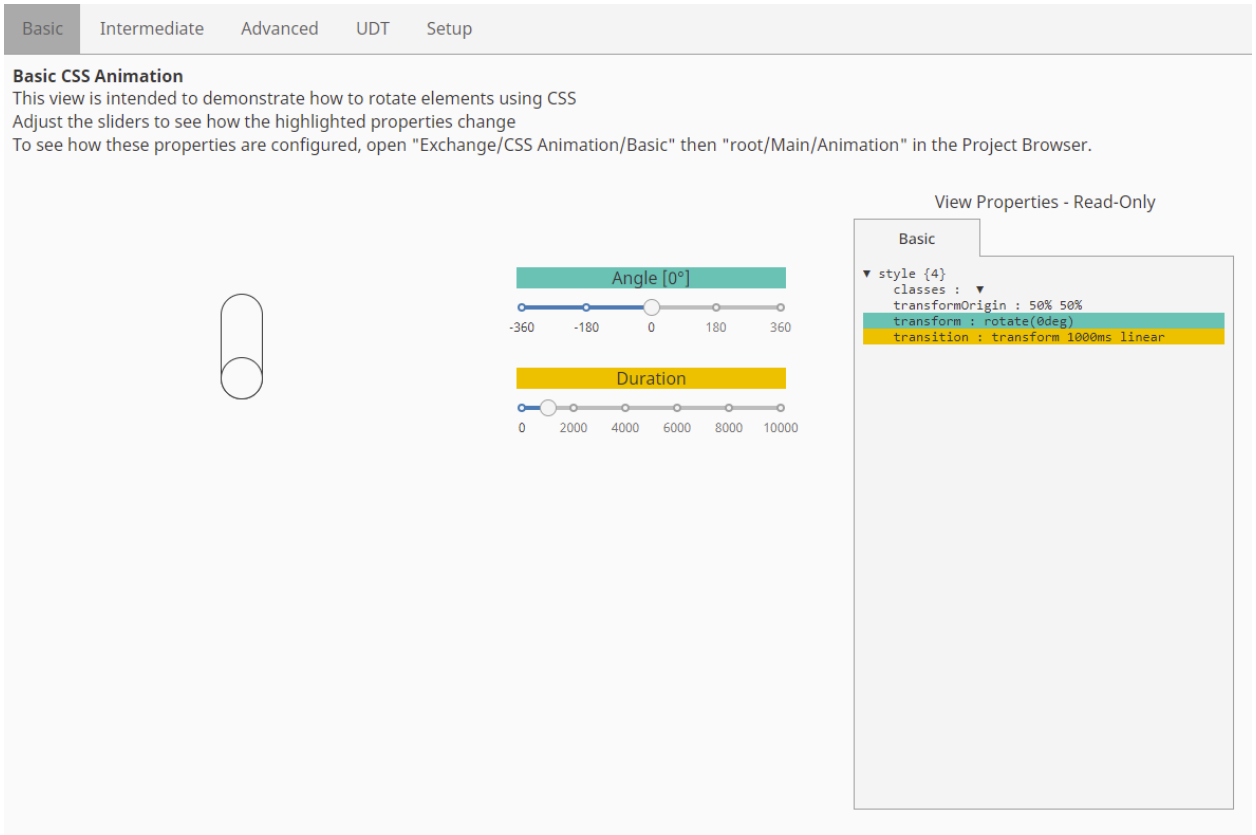
Intermediate....4
- Properties....4
- In Designer....5

Advanced....9
- In Designer....10
  - Arm View Parameters....10
  - Arm View Custom Properties....10
  - Command Property....11
  - Robot Simple View....13

User Defined Types - UDTs....14
- In Designer....15
  - Joint UDT....16
  - StateMachine UDT....19
  - RobotAnimation UDT....24

Setup....26
- Animation....27
  - Settings and XY Motion....28
- Browse Tags....29
- Steps....30
- Interlocks....33
- Tag Writes....34
- Permissives....35

Basic



Properties

transformOrigin

50% 50% - location around which the container will be rotated.
In this case it is rotated around the center.
Additional info: https://www.w3schools.com/cssref/css3_pr_transform-origin.php

transform

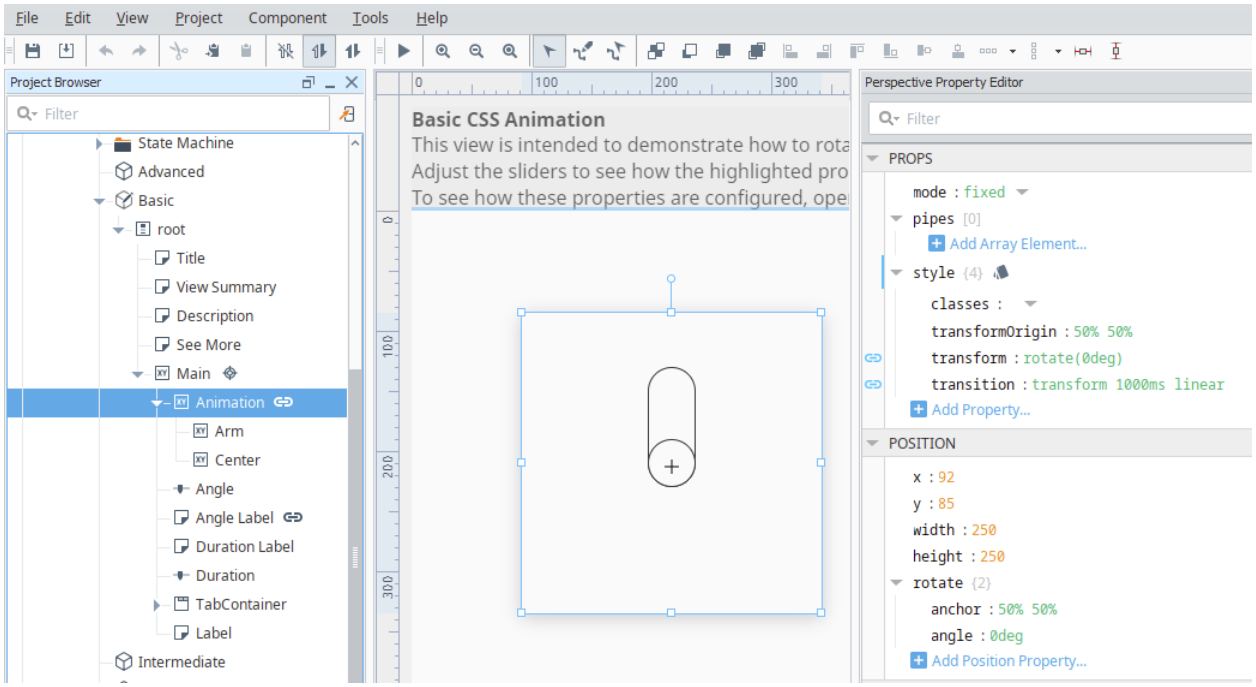
rotate(0deg) - rotation from the original position around the transformOrigin.
Other transformations are also supported and can be found in the link below.
Additional info: https://www.w3schools.com/cssref/css3_pr_transform.php

transition

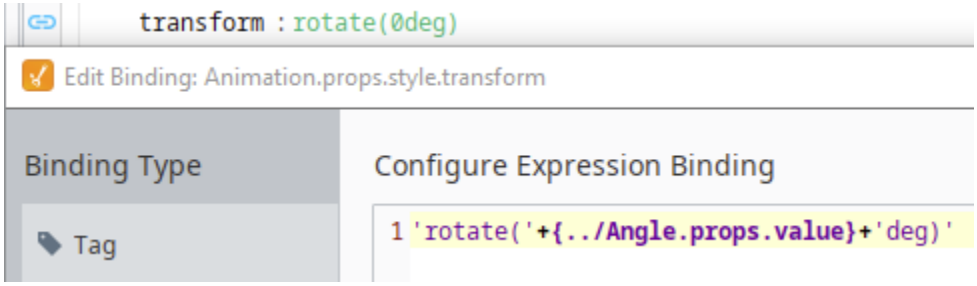
transform 1000ms linear - executes transform defined above over specified duration, using linear timing function. Linear is used for this demo to simplify pausing logic.
Additional info: https://www.w3schools.com/cssref/css3_pr_transition.php

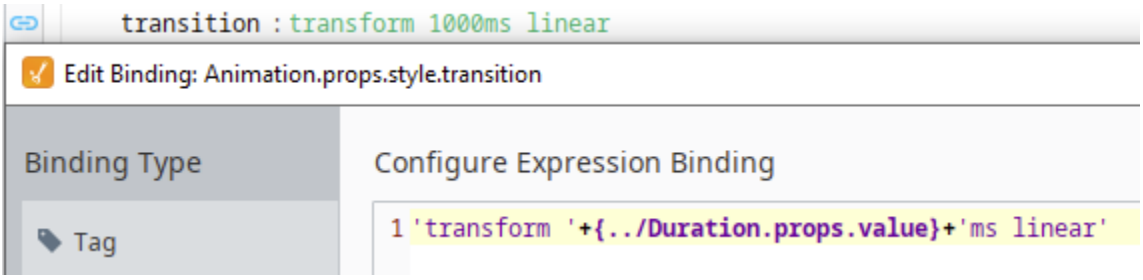
In Designer

To see how these properties are configured, open "Exchange/CSS Animation/Basic" then "root/Main/Animation" in the Project Browser.

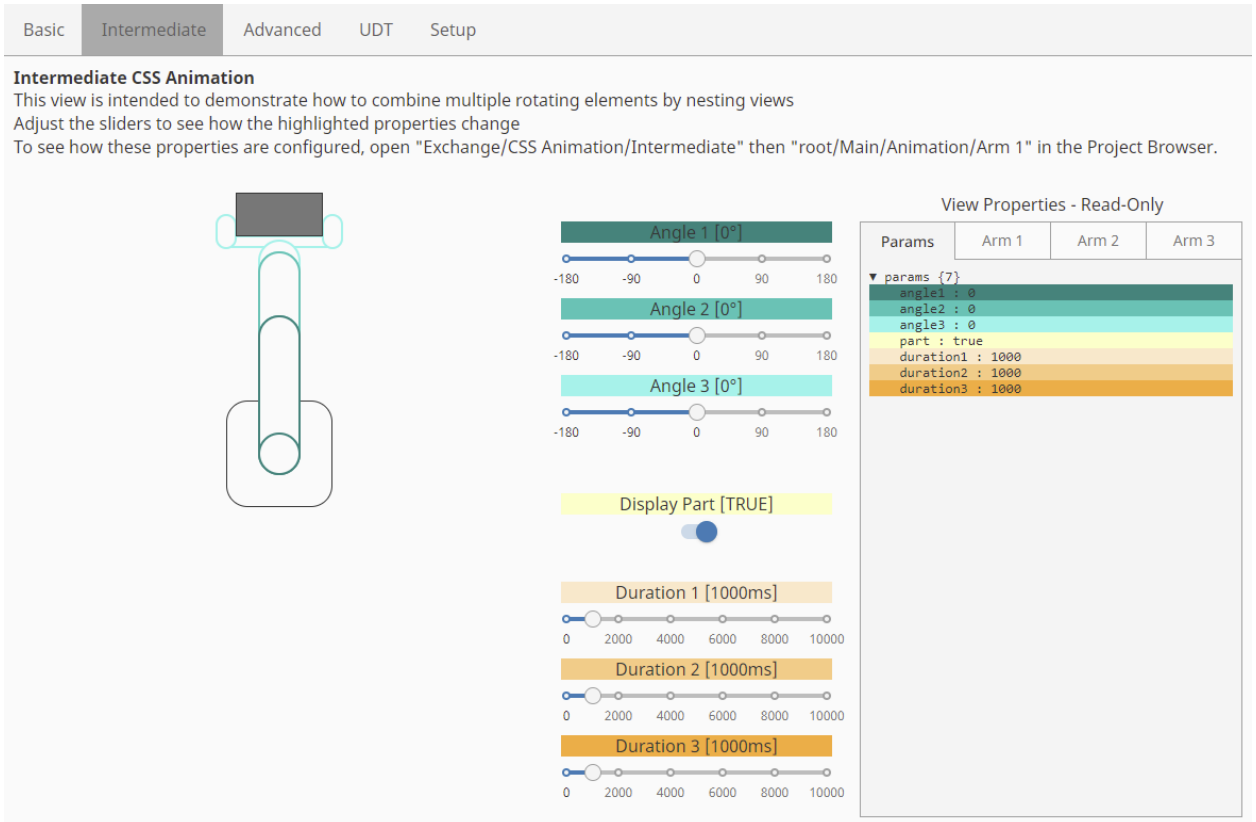


There are two string wrapping expressions that are binded to the slider values.





Intermediate

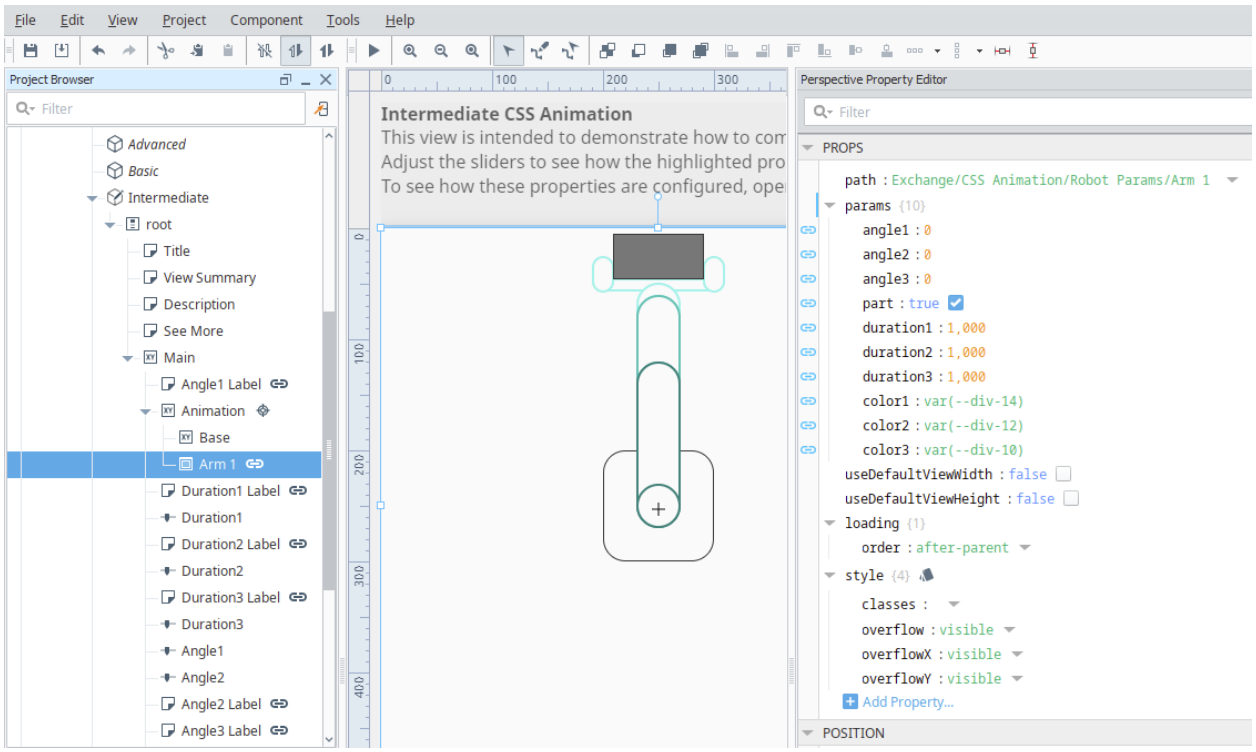


Properties

angle1: 0 - angle of the first arm, attached to the base
angle2: 0 - angle of the second arm, attached to the first arm
angle3: 0 - angle of the third arm containing the end of arm and the part, attached to second arm
part: true - whether or not the part should be displayed
duration1: 1000 - duration in milliseconds of first arm animation
duration2: 1000 - duration in milliseconds of second arm animation
duration3: 1000 - duration in milliseconds of third arm animation

In Designer

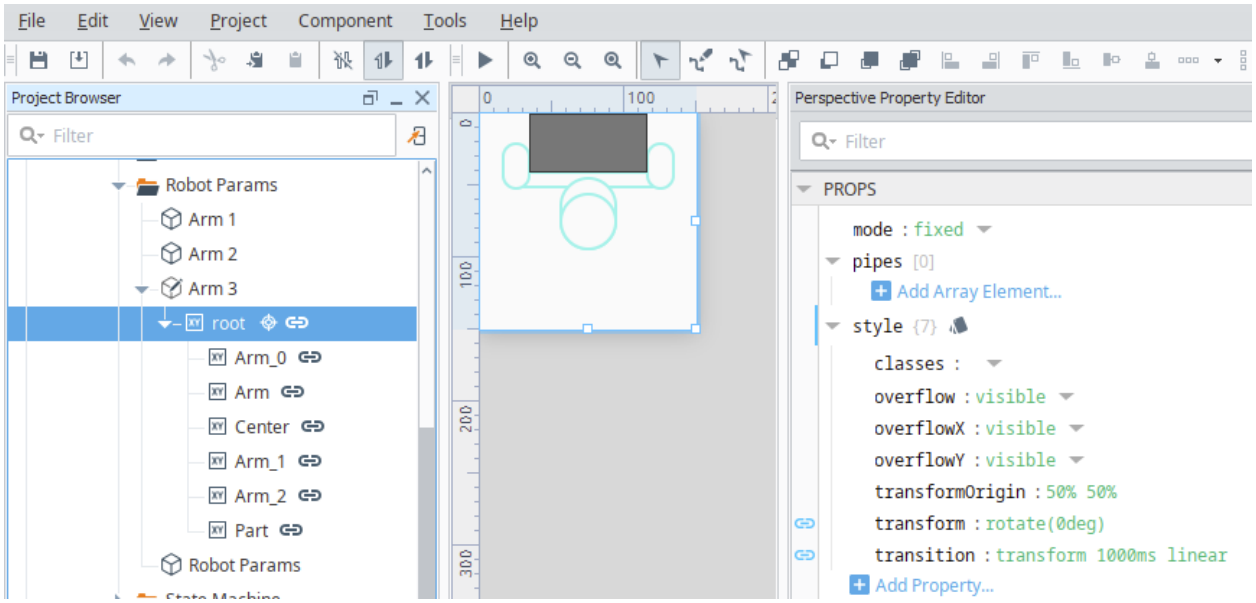
To see how these properties are configured, open "Exchange/CSS Animation/Intermediate" then "root/Main/Animation/Arm 1" in the Project Browser.



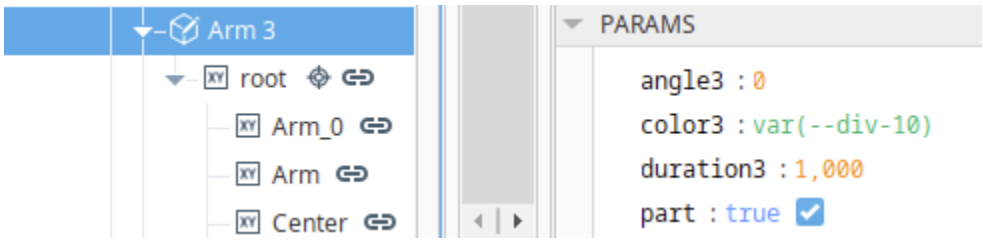
The properties are binded to the sliders and the toggle similar to the Basic example.
Color properties are only used for this intermediate portion to more easily visualize bindings.
In this case we are embedding another view located at *Exchange/CSS Animation/Robot Params/Arm 1* but to understand how things are layered we will start with the most basic: Arm 3.
Coordinate containers are used to make these animations as they have the most predictable aspect ratios, which is necessary for complex models.

Exchange/CSS Animation/Robot Params/Arm 3

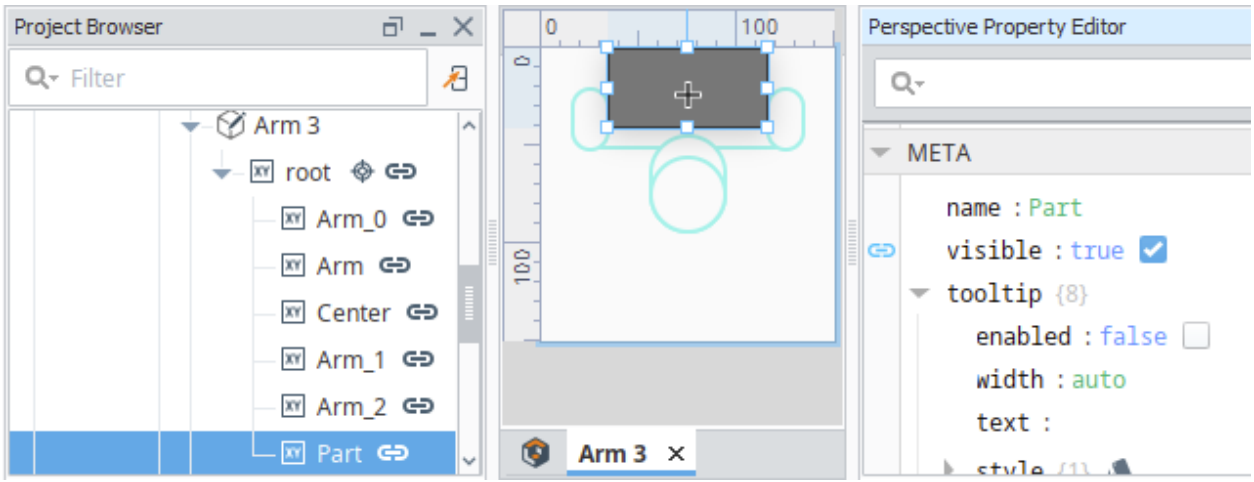
This view is very similar to the Basic example. It uses input parameters to generate transform and transition. Overflow is set to visible so components are still seen if the object is rotated out of the initial container. Tip: creating a *Center* object makes future alignment easier.



Input Parameters:

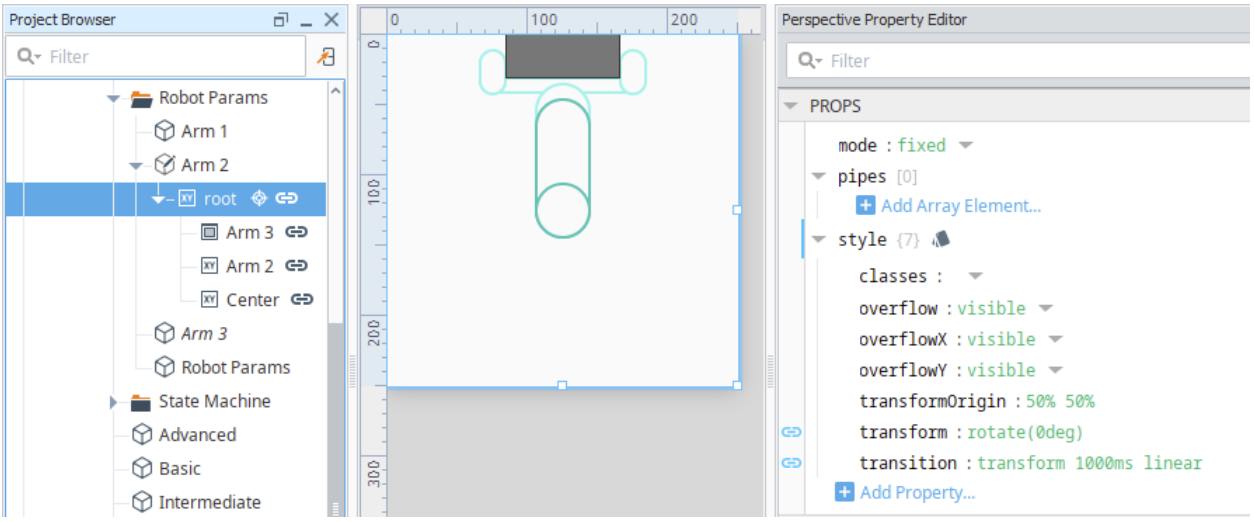


Part visibility is binded to the input parameter:

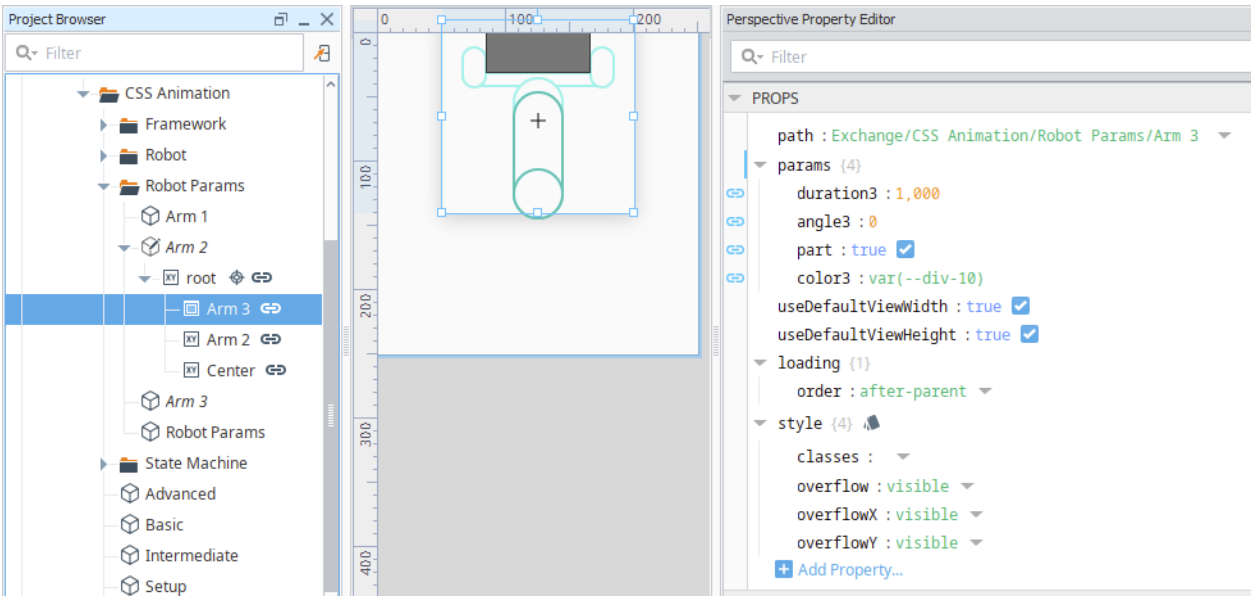


Exchange/CSS Animation/Robot Params/Arm 2

Arm 2 is similar to Arm 3 at the root container level

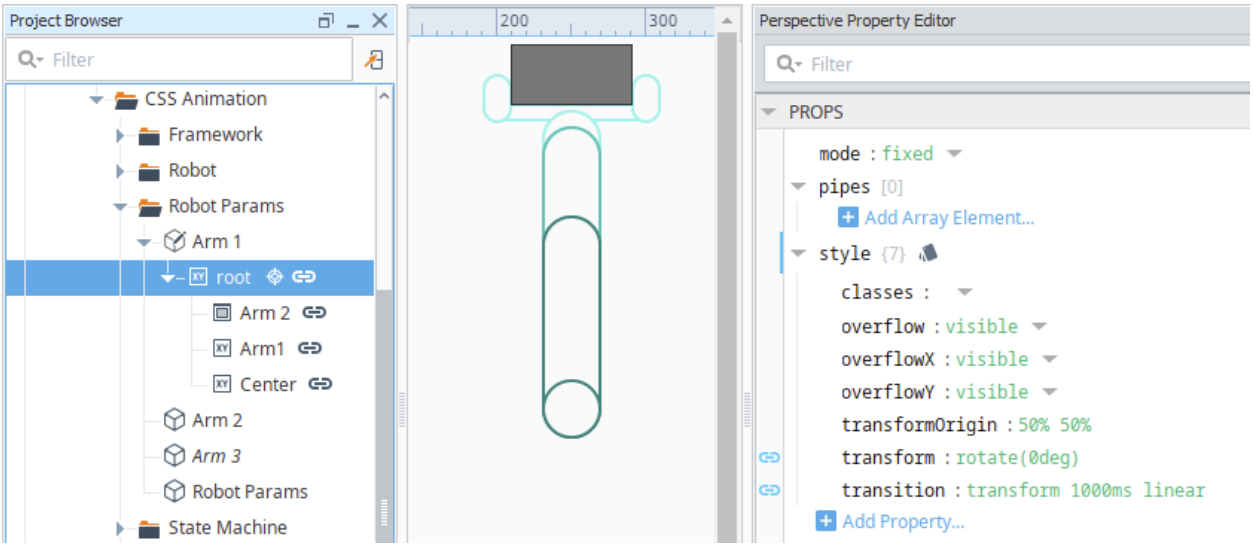


To achieve multi-joint effect Arm 3 view is embedded into Arm 2. This means angle3 is always in reference to angle 2.
Tip: It is critical for the center of Arm 3 to align with the location on Arm 2 where the joint should be. Temporarily layering Arm 3 on top and showing the *Center* component can be very useful here.

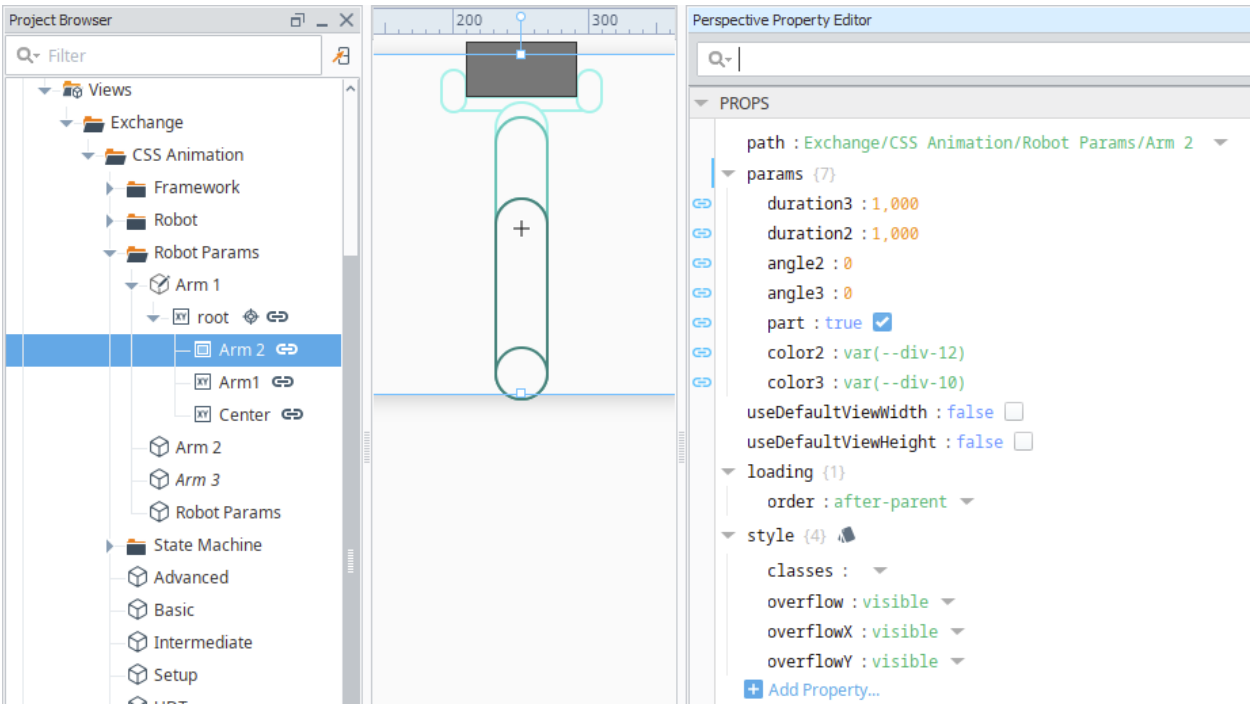


Exchange/CSS Animation/Robot Params/Arm 1

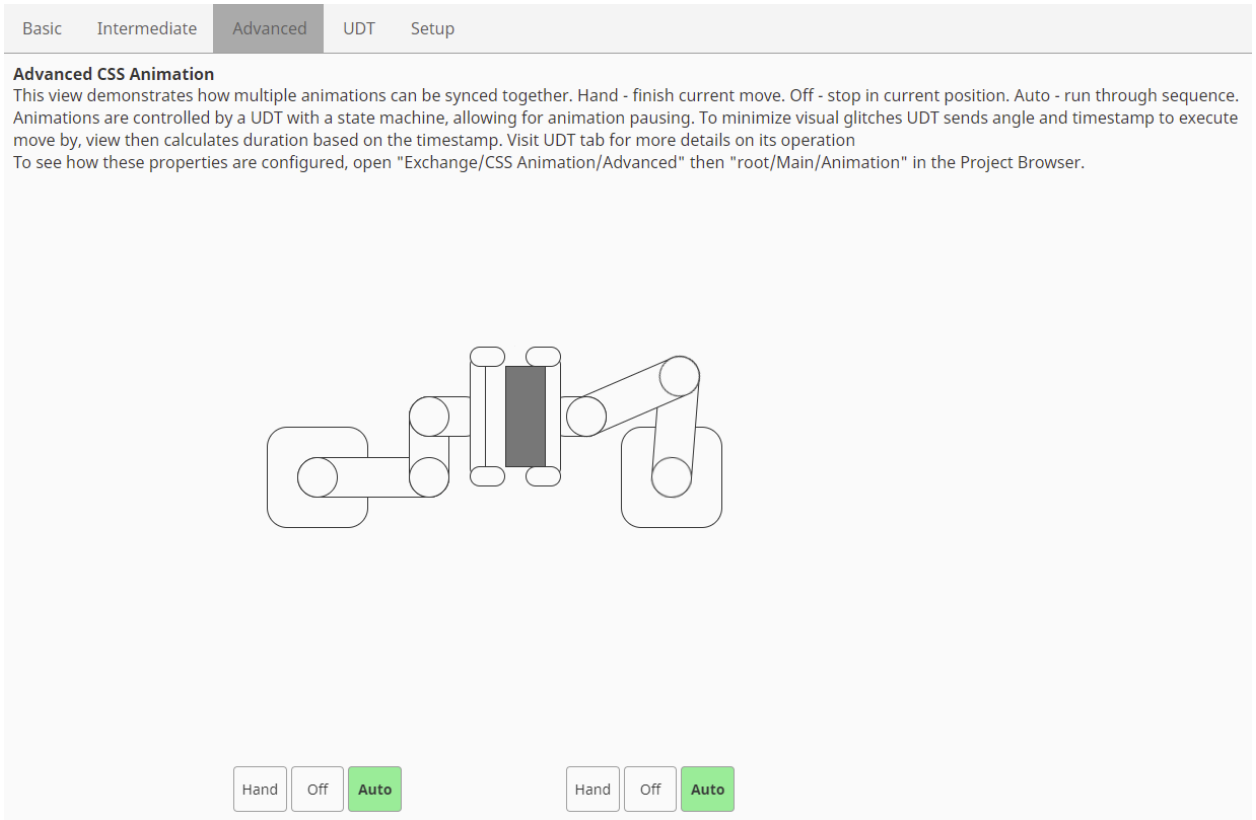
Arm 1 is very similar to Arm 2 in composition



Arm 2 is embedded into Arm 1 to achieve the multi joint effect, just like Arm 2.
Similarly angle2 is always in reference to angle1.



Advanced

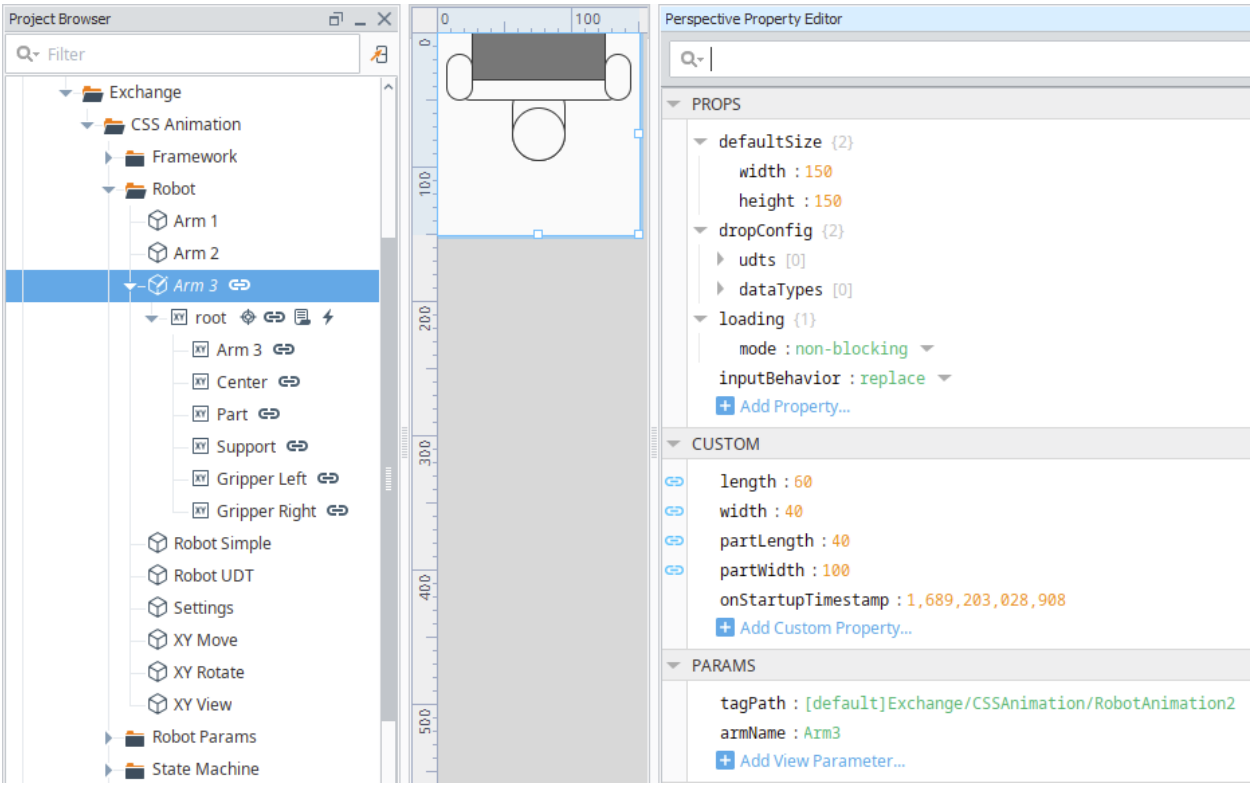


In order to animate multiple robots on screen changes are made to the intermediate example.
To allow for pausing of animation a UDT that calculates angle to stop at was introduced.
Length and width of moving parts was made dynamic, controlled by UDT.
To reduce glitches due to views having variable load time a new command property was added.

In Designer

Before opening these views it is recommended to switch RobotAnimation2 into hand control and command all angles to 0. Otherwise containers could continue moving or be in awkward positions, hindering structure comprehension.

To see how these properties are configured, open "Exchange/CSS Animation/Advanced" then "root/Main/Animation" in the Project Browser.



Arm View Parameters

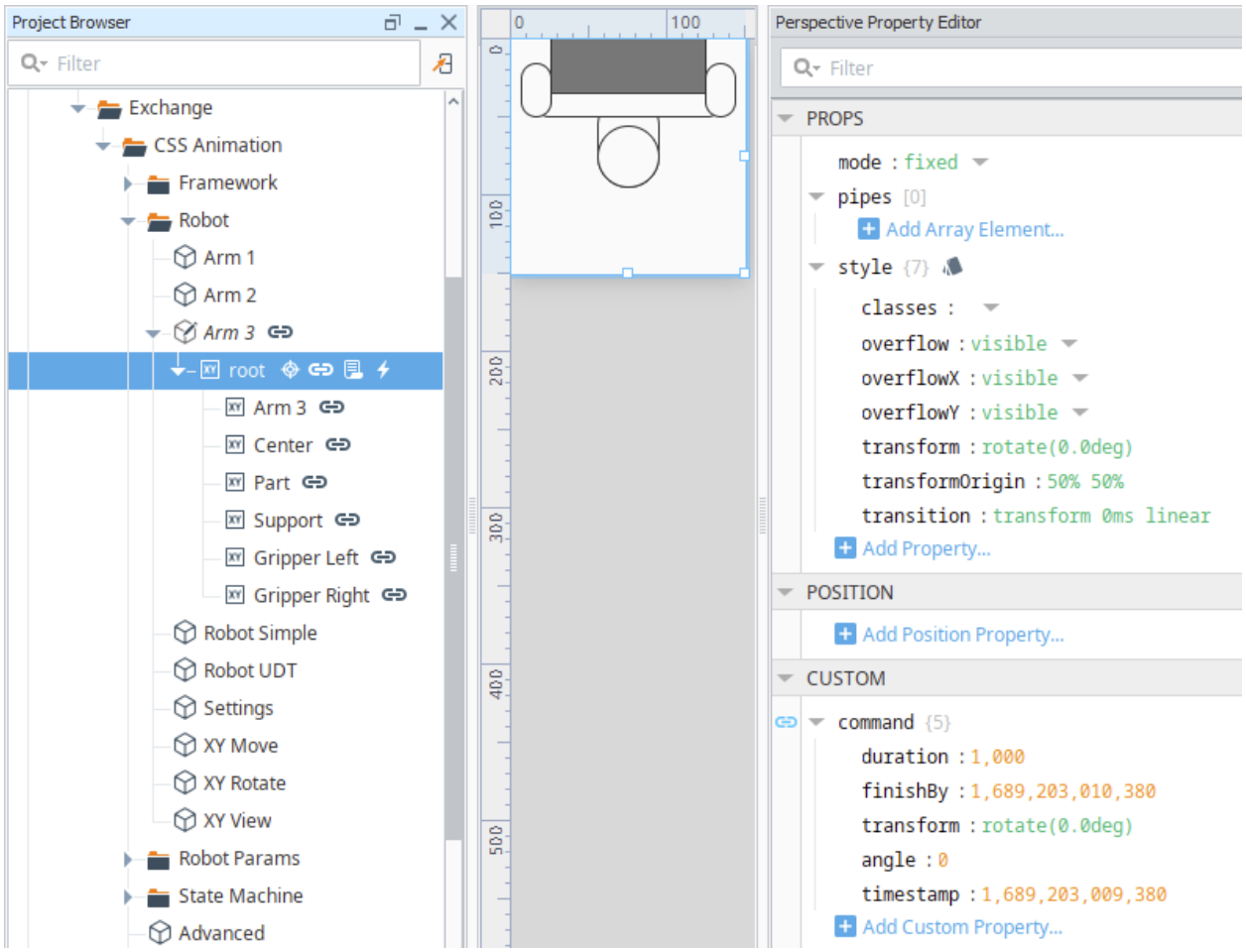
tagPath - path to the UDT
armName - arm name, used to make full tag path to required tags

Arm View Custom Properties

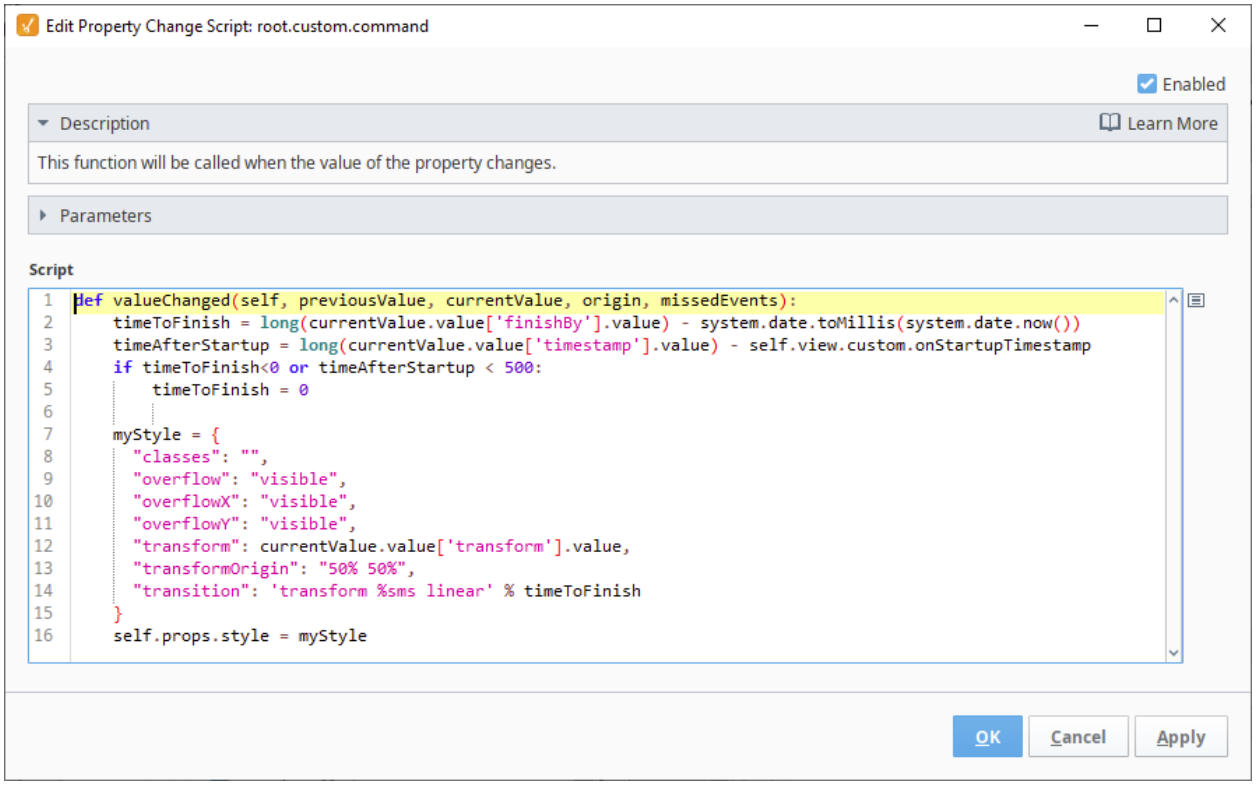
length - length of the robot arm
width - width of the robot arm
partLength - length of the part
partWidth - width of the part
onStartupTimestamp - set by the onStartup system event of the root container.
Used in [Command Property Change Script]() to avoid showing joints in incorrect position

Command Property

To avoid race conditions with using multiple tags the UDT packages the most important values into one document tag named Command. The tag is binded to a custom property with a Property Change Script that updates the entire style of the root container. Note there are no more bindings on the transform and transition properties. For more information on the Command tag see the [UDT section](#).



Command Property Change Script



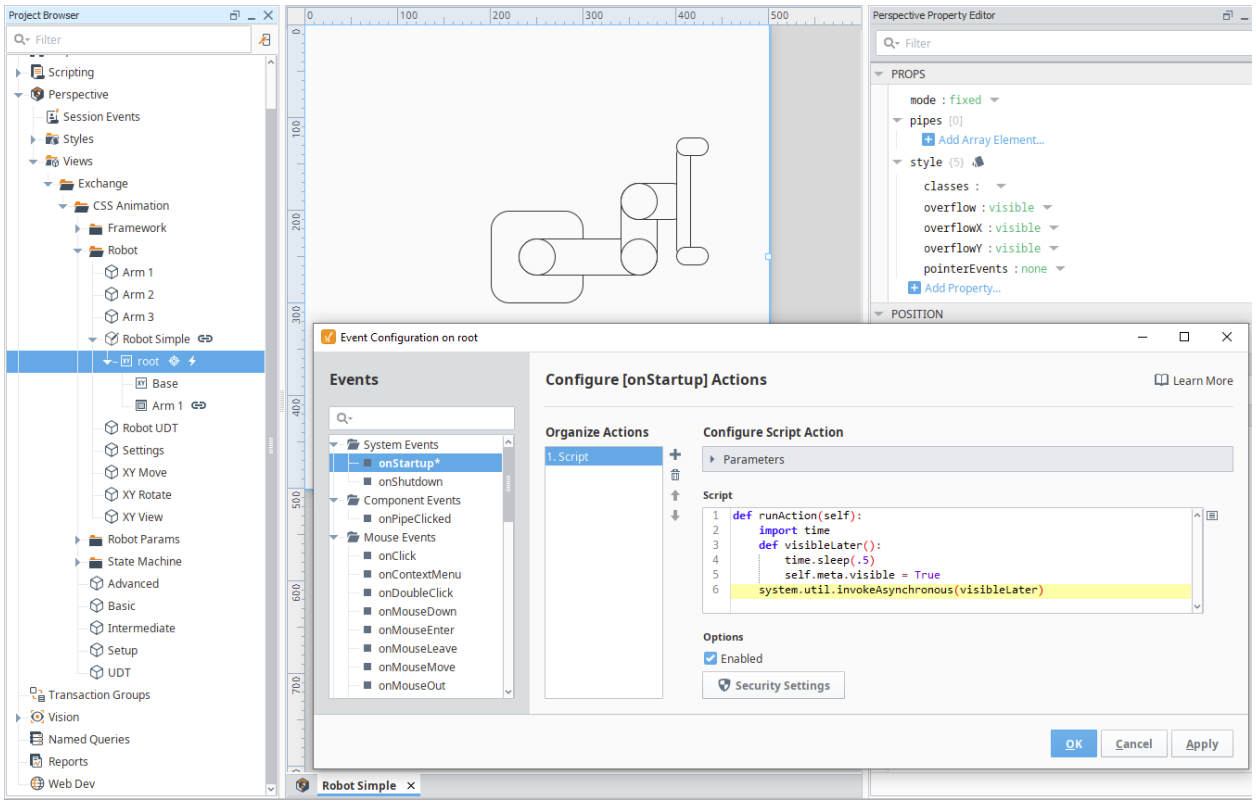
timeToFinish - this value is calculated locally in a client based on when the view is loaded

timeAfterStartup - used to complete animation instantly if the view loaded within last 500 milliseconds

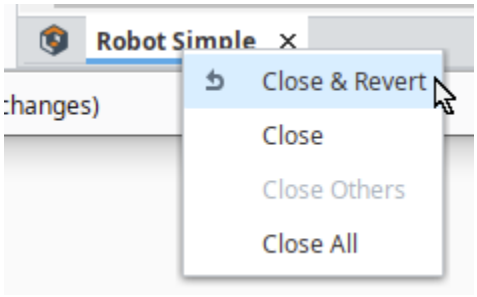
myStyle - one property to be set to avoid race conditions between transform and transition changing at slightly different times and causing glitchy behavior

Robot Simple View

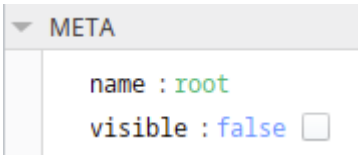
This view adds a base graphic and has an onStartup script that has a half second delay. When the view loads the components will be in initial positions where they were saved in the designer, then move to the angle specified by the command. The loading process can make components look in the wrong position, to mitigate that the whole container is hidden for the first half second and then shown by the script.



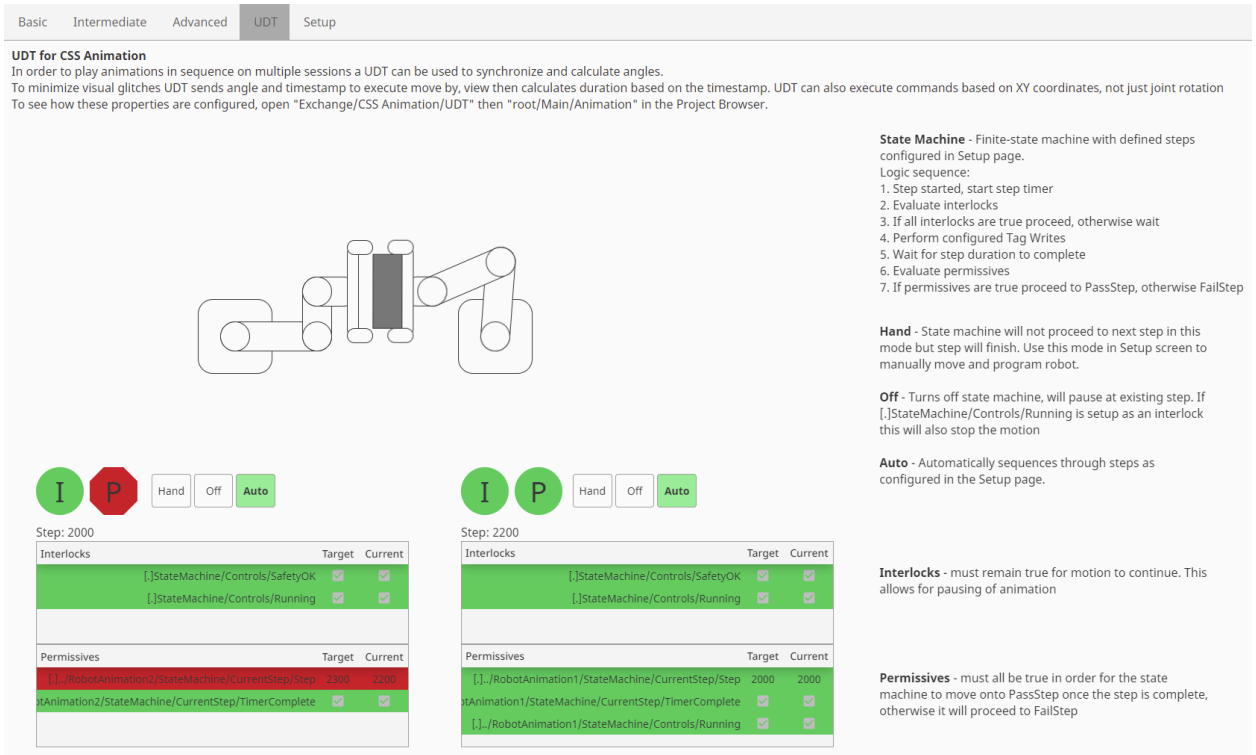
Note: it is important to save this view with self.meta.visible set to false, otherwise functionality will not work. Opening this view in designer triggers the onStartup script and shows the container. If not making any changes close this view with Close & Revert. If making changes make sure to uncheck the visible property on the root container before saving.



OR

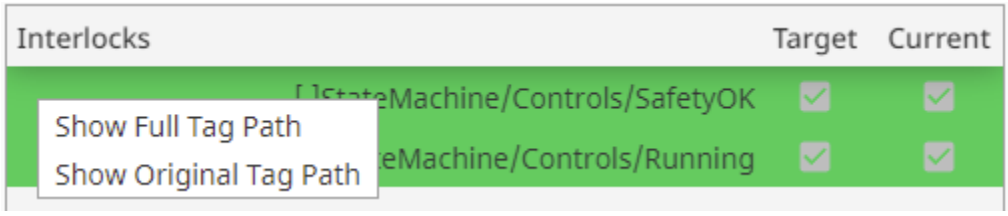


User Defined Types - UDTs



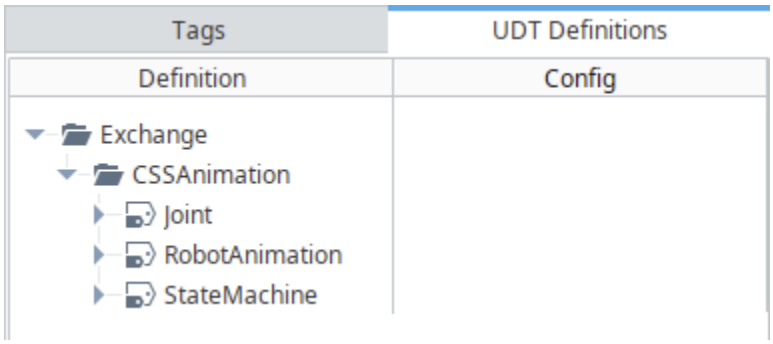
The UDT has several important features. It serves as a source of truth for angles of the robot so multiple sessions can show the same animation. Including a pause functionality that allows robots to stop at an angle that is not the current destination. Most importantly the UDT has a state machine that can sequence one animation after another and even sync multiple UDTs together.

Right clicking on interlocks or permissives brings up a context menu which can change how the tag paths are displayed.



In Designer

There are three different UDTs:



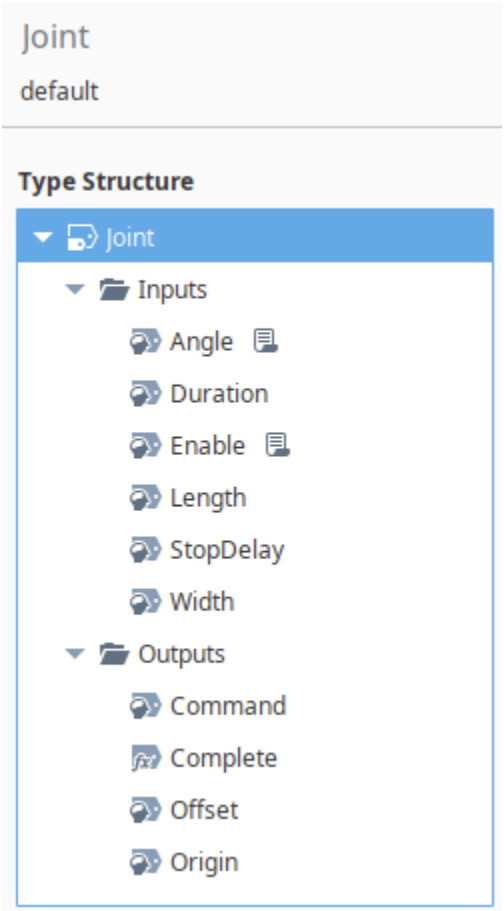
Joint - handles animation of a single joint or an arm

RobotAnimation - incorporates Joint and StateMachine UDTs to perform the full robot animation

StateMachine - controls sequence using tags

Joint UDT

Handles animation of a single joint or an arm



- **Inputs**
  - **Angle** - Value of current angle, see script.
  - **Duration** - Value of duration in milliseconds.
  - **Enable** - Determines if animation should be running or not, see script.
  - **Length** - Length of the arm.
  - **StopDelay** - Delay in milliseconds before animation comes to a stop.
  - **Width** - Width of the arm.
- **Outputs**
  - **Command** - Value is generated by Angle and Enable tag scripts. By having command be one document tag many race conditions are eliminated where angle but not duration are not updated at exact same time.
  - **Complete** - Determines if animation finished.
  - **Offset** - Updated by Enable tag to store progress if paused.
  - **Origin** - Previous value of Angle, used to calculate where to pause.

Angle Tag Event Script

Script

```
def valueChanged(tag, tagPath, previousValue, currentValue, initialChange, missedEvents):
    tags = system.tag.readBlocking([
        "[.]../Outputs/Command"
        ,"[.]Duration"
    ])
    command  = tags[0]
    duration = tags[1].value

    newCommand = {
        'finishBy': system.date.toMillis(currentValue.timestamp) + duration
        ,'transform':'rotate(%sdeg)'%currentValue.value
        ,'angle': currentValue.value
        ,'duration': duration
        ,'timestamp' : system.date.toMillis(currentValue.timestamp)
    }

    tagPaths = ["[.]../Outputs/Origin","[.]../Outputs/Command","[.]../Outputs/Offset","[.]Enable"]
    values = [command.value.angle,system.util.jsonEncode(newCommand),0,1]
    system.tag.writeBlocking(tagPaths, values)
```

Compiles **newCommand** from required UDT tags. By having command be one document tag many race conditions are eliminated.

Updates Origin to value previously held in the Command tag.
Writes newCommand into Command.
Resets Offset down to zero.
Enables animation.

Enable Tag Event Script

Script

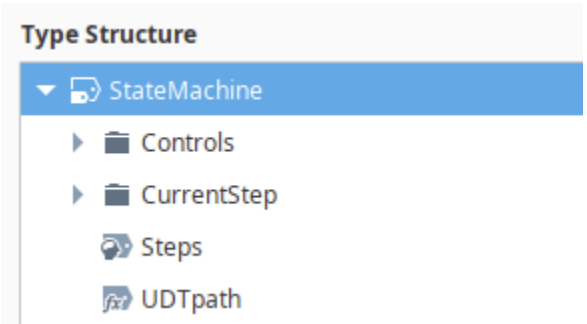
```python
def valueChanged(tag, tagPath, previousValue, currentValue, initialChange, missedEvents):
    tags = system.tag.readBlocking([
        "[.]../Outputs/Command"
        ,"[.]StopDelay"
        ,"[.]Duration"
        ,"[.]Angle"
        ,"[.]../Outputs/Offset"
        ,"[.]../Outputs/Origin"
    ])
    command   = tags[0]
    stopDelay = tags[1].value
    duration  = tags[2].value
    angle     = tags[3].value
    offset    = tags[4].value
    origin    = tags[5].value

    if currentValue.value == False:
        lastAction = max(command.timestamp, previousValue.timestamp)

        offset = offset + stopDelay + system.date.millisBetween(lastAction, currentValue.timestamp)
        if offset>=duration:
            offset = duration
            pauseAngle = angle
        else:
            pauseAngle = origin + 1.0*offset/duration*(angle-origin)

        newCommand = {
            'finishBy': system.date.toMillis(currentValue.timestamp) + stopDelay
            ,'transform':'rotate(%sdeg)'%pauseAngle
            ,'angle': pauseAngle
            ,'duration': stopDelay
            ,'timestamp' : system.date.toMillis(currentValue.timestamp)
        }
    else:
        newCommand = {
            'finishBy': system.date.toMillis(currentValue.timestamp) + duration - offset
            ,'transform':'rotate(%sdeg)'%angle
            ,'angle': angle
            ,'duration': duration - offset
            ,'timestamp' : system.date.toMillis(currentValue.timestamp)
        }

    tagPaths = ["[.]../Outputs/Command","[.]../Outputs/Offset"]
    values = [system.util.jsonEncode(newCommand),offset]
    system.tag.writeBlocking(tagPaths, values)
```

If the Enable tag is **turned off** the script calculates offset and pauseAngle, then compiles a newCommand document where finishBy is the current timestamp plus stopDelay. Since tags are executed on the gateway there could be a slight lag between session and the gateway. That lag could result in a pause angle to be behind where the animation already traveled. By stopping it in the future that glitch is eliminated.

If the Enable tag is **turned on** a newCommand is generated using current duration and offset values. This allows for resuming of the animation.

StateMachine UDT

Finite-state machine UDT that allows for sequencing of actions



- **Controls**
  - **HOA** - Control values, Hand-2 / Off-0 / Auto-1
  - **Running** - On when HOA is not off.
  - **SafetyOK** - Safety input, stop motion if used as an interlock.
- **CurrentStep**
  - **DurationMS** - Duration of the step.
  - **FailStep** - Step to which state machine will proceed if permissives are not met by the time the step timer finishes. If 0 the state machine will remain on the current step.
  - **Interlocks** - Dataset of interlock tags that must be true for a state machine to run.
  - **InterlocksOK** - All interlocks are satisfied or empty.
  - **LastStep** - Previous step of the state machine, used to determine when a new step is started.
  - **Name** - Name of the current step.
  - **PassStep** - Step to which state machine will proceed if permissives are met by the time the step timer finishes.
  - **Permissives** - Dataset of permissive tags that must be true for a state machine to proceed to PassStep.
  - **PermissivesOK** - All permissives are satisfied or empty.
  - **Script** - Main script where logic occurs.
  - **Status** - Status of the current step.
  - **Step** - Current step.
  - **TagWrites** - Dataset of tag paths that will be executed during this step.
  - **TagWritesOK** - Tag writes were completed successfully.
  - **Timer** - How long has a specific step been running, paused with interlocks are not ok.
  - **TimerComplete** - Boolean for when the timer has finished.
  - **TimerOffset** - Timer offset used for pausing and resuming a step.
  - **TimerTimestamp** - Tag used by Timer as the starting timestamp. It is written by Script and InterlocksOK tag change scripts.
- **Steps** - Dataset of all steps in this state machine.
- **UDTpath** - Path of this UDT, used for dynamic tag paths.

InterlocksOK Tag Event Script

Script

```
def valueChanged(tag, tagPath, previousValue, currentValue, initialChange, missedEvents):
    if currentValue.value == False:
        tagPaths = ["[.]TimerOffset","[.]Step"]
        tags = system.tag.readBlocking(tagPaths)
        offset = tags[0].value
        step = tags[1]
        timestamp = max(step.timestamp, previousValue.timestamp)

        tagPaths = ["[.]TimerOffset","[.]TimerTimestamp"]
        values = [offset + system.date.millisBetween(timestamp, system.date.now()), currentValue.timestamp]
        system.tag.writeBlocking(tagPaths, values)
    else:
        tagPaths = ["[.]TimerTimestamp"]
        values = [currentValue.timestamp]
        system.tag.writeBlocking(tagPaths, values)
```

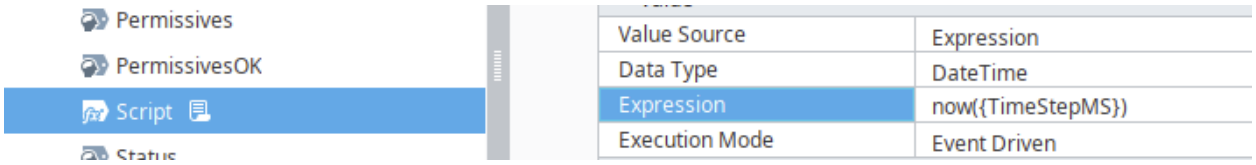
This logic allows for pausing and resuming of the state machine Timer.

If the tag is turned off new TimerOffset is calculated and TimerTimestamp is updated.
If the tag is turned on the TimerTimestamp is updated.

Script Tag Event Script

Due to the length of the script only key elements are explained.

This script executes every 100ms by default.
The value can be changed by the TimeStepMS parameter.



The script starts out by reading all required UDT tags.

```
7     rwPaths = [
8         "[.]Interlocks"
9        ,"[.]InterlocksOK"
10       ,"[.]TagWrites"
11       ,"[.]TagWritesOK"
12       ,"[.]Permissives"
13       ,"[.]PermissivesOK"
14       ,"[.]Status"
15       ,"[.]Step"
16       ,"[.]LastStep"
17    ]
18    roPaths = [
19        "[.]TimerComplete"
20       ,"[.]PassStep"
21       ,"[.]FailStep"
22       ,"[.]../Controls/HOA"
23       ,"[.]../UDTpath"
24    ]
25    tags = system.tag.readBlocking(rwPaths+roPaths)
```

If the step was changed it will load new values into Interlocks, TagWrites, and Permissives.

```
46    if lastStep != step:
47        lastStep = step
...
86                interlocks    = columnToDataset('Interlocks')
87                interlocksOK  = False
88                tagWrites     = columnToDataset('TagWrites')
89                tagWritesOK   = False
90                permissives   = columnToDataset('Permissives')
91                permissivesOK = False
92                timerComplete = False
```

The script reads all Interlocks and Permissives and puts them into a dictionary for later use.

```
# compile a list of all tags to read, interlocks + permissives
tagPaths=[]
for i in range(interlocks.getRowCount()):
    tagPaths.append(interlocks.getValueAt(i,'tagPath'))
for i in range(permissives.getRowCount()):
    tagPaths.append(permissives.getValueAt(i,'tagPath'))
tagPaths = list(set(tagPaths)) # removes duplicates
tagDict = {}
if tagPaths:
    tagValues = system.tag.readBlocking(tagPaths)
    for i, tagPath_ in enumerate(tagPaths):
        tagDict.update({tagPath_:tagValues[i].value})
```

If Interlocks are satisfied it will execute all of TagWrites.

```
# common function used by interlocks and permissives
def checkTags(dataset):
    allOK = True
    for i in range(dataset.getRowCount()):
        currValue = tagDict[dataset.getValueAt(i,'tagPath')]
        dataset = system.dataset.setValue(dataset, i,'currentValue',currValue)
        if str(currValue).upper() != str(dataset.getValueAt(i,'tagValue')).upper():
            allOK = False
    return dataset, allOK

# check interlocks, pause if not ok
interlocks, interlocksOK = checkTags(interlocks)

# execute tag writes once
writePaths = []
writeValues = []
if not tagWritesOK and interlocksOK and HOA != 0:
    for i in range(tagWrites.getRowCount()):
        writePaths.append(tagWrites.getValueAt(i,'tagPath'))
        writeValues.append(tagWrites.getValueAt(i,'tagValue'))
    tagWritesOK = True
    status = {'Running':'Executed tag writes for step %s'%step}
```

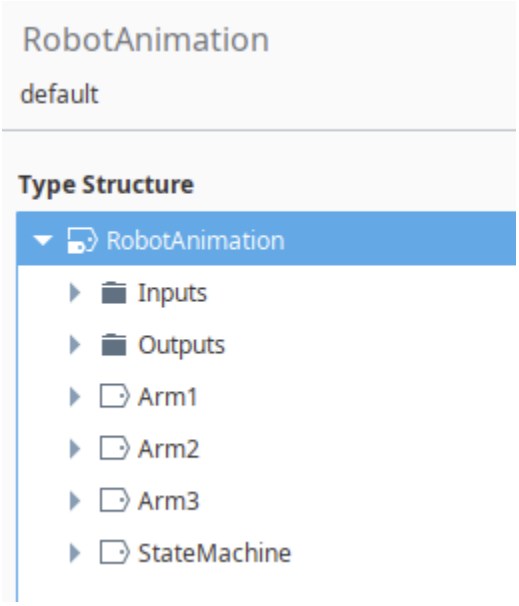
If Interlocks are ok, tag writes finished and the step timer is complete the script will proceed to next step selection. If all Permissives are satisfied and the state machine is in Auto it will proceed to PassStep. If not and FailStep is defined (greater than 0) it will proceed to FailStep.

```
149     # check permissives, PassStep if ok, FailStep if not ok
150     permissives, permissivesOK = checkTags(permissives)
151
152     if not interlocksOK:
153         status = {'Idle':'Waiting for interlocks, step %s'%step}
154     elif not tagWritesOK:
155         status = {'Idle':'Waiting for tag writes, step %s'%step}
156     else:
157         if timerComplete:
158             if permissivesOK:
159                 if HOA == 1: step = passStep
160             else:
161                 if failStep > 0:
162                     status = {'Faulted':'Waiting for permissives, step %s'%step}
163                     if HOA == 1: step = failStep
164                 else:
165                     status = {'Idle':'Waiting for permissives, step %s'%step}
```

Newly updated tag values are written back into the UDT.

```
170     rwValues = [
171         interlocks
172         ,interlocksOK
173         ,tagWrites
174         ,tagWritesOK
175         ,permissives
176         ,permissivesOK
177         ,newStatus
178         ,step
179         ,lastStep
180     ]
181
182     system.tag.writeBlocking(rwPaths+stepChangePaths+writePaths, rwValues+stepChangeValues+writeValues)
```

RobotAnimation UDT



- **Inputs**
  - **ExecuteXY** - Used to re-trigger /Output/Command when X,Y,Theta have not changed.
  - **Part** - Boolean to display part or not.
  - **PartLength** - Length of the part.
  - **PartWidth** - Width of the part.
  - **Theta** - Angle of the part during XY move.
  - **X** - X location for the XY move.
  - **Y** - Y location for XY move.
- **Outputs**
  - **Command** - Combination of X, Y, Theta, ExecuteXY tags to avoid race conditions. This tag's event script calculates and sends appropriate angles to all of the joints within this UDT.
  - **J3X** - Joint 3 X current position, used in Command calculations.
  - **J3Y** - Joint 3 Y current position, used in Command calculations.
  - **PartAngle** - Current part angle.
  - **PartCenterX** - Current X location of the part center.
  - **PartCenterY** - Current Y location of the part center.
- **Arm1** - Joint UDT.
- **Arm2** - Joint UDT.
- **Arm3** - Joint UDT.
- **StateMachine** - StateMachine UDT.

Command Tag Event Script

Due to the length of the script only key elements are explained.

Main purpose of this script is to calculate individual joint angles needed to achieve requested X, Y, Theta positions. Since the Theta is known the problem can be simplified to calculate only joint1 and joint2 angles as joint3 can be rotated to any angle to achieve the Theta.

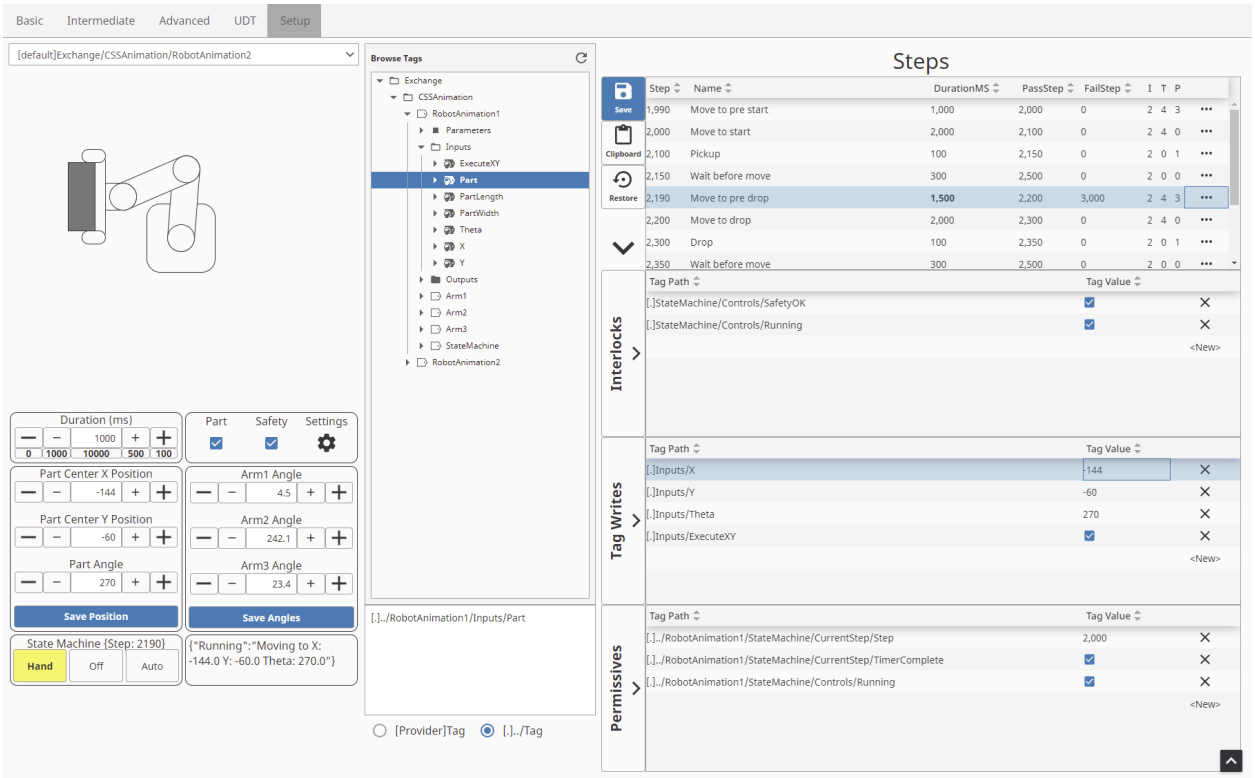
Following function getJointAngles takes in lengths of arm1 and arm2 and the desired x, y coordinates and outputs joint angles.

```
def getJointAngles(len1, len2, x, y):
    lenXY = (x**2+y**2)**0.5 # length of third imaginary edge at x,y location
    if lenXY==len1+len2:
        angleXY = 180
        angle2 = 0
    else:
        angleXY = math.degrees(math.acos((len1**2+len2**2-lenXY**2)/(2*len1*len2))) # law of cosine, angle opposing lenXY
        angle2 = math.degrees(math.asin(len2/lenXY*math.sin(math.radians(angleXY)))) # law of sine, angle opposing len2
    alpha = math.degrees(math.asin(y/lenXY)) # angle from x plane
    if shortestAngle(arm2current,0) >=0:
        if x>=0:
            joint1    = 90-alpha-angle2
        else:
            joint1    = -90+alpha-angle2
        joint2 = 180-angleXY
    else:
        if x>=0:
            joint1 = 90-alpha+angle2
        else:
            joint1 = -90+alpha+angle2
        joint2 = -180+angleXY
    return joint1%360, joint2%360
```

If the move is possible script will send new tag values to all 3 Joint UDTs

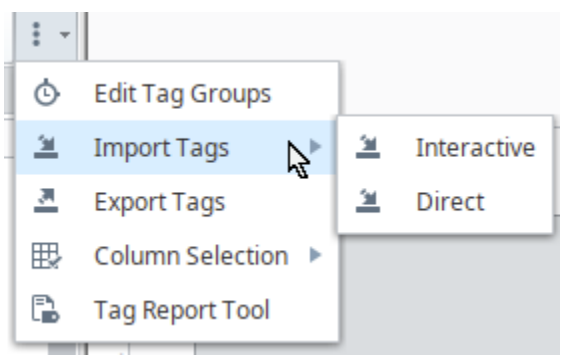
```
tagPaths = [
    "[.]../Inputs/ExecuteXY"
    ,"[.]../Arm1/Inputs/Angle"
    ,"[.]../Arm2/Inputs/Angle"
    ,"[.]../Arm3/Inputs/Angle"
    ,"[.]../StateMachine/CurrentStep/Status"
]

values = [
    False
    ,udt.Arm1.Inputs.Angle + arm1delta
    ,udt.Arm2.Inputs.Angle + arm2delta
    ,udt.Arm3.Inputs.Angle + arm3delta
    ,system.util.jsonEncode({'Running':'Moving to X: %s Y: %s Theta: %s'%(udt.Inputs.X, udt.Inputs.Y, udt.Inputs.Theta)})
]

system.tag.writeBlocking(tagPaths, values)
```

Setup

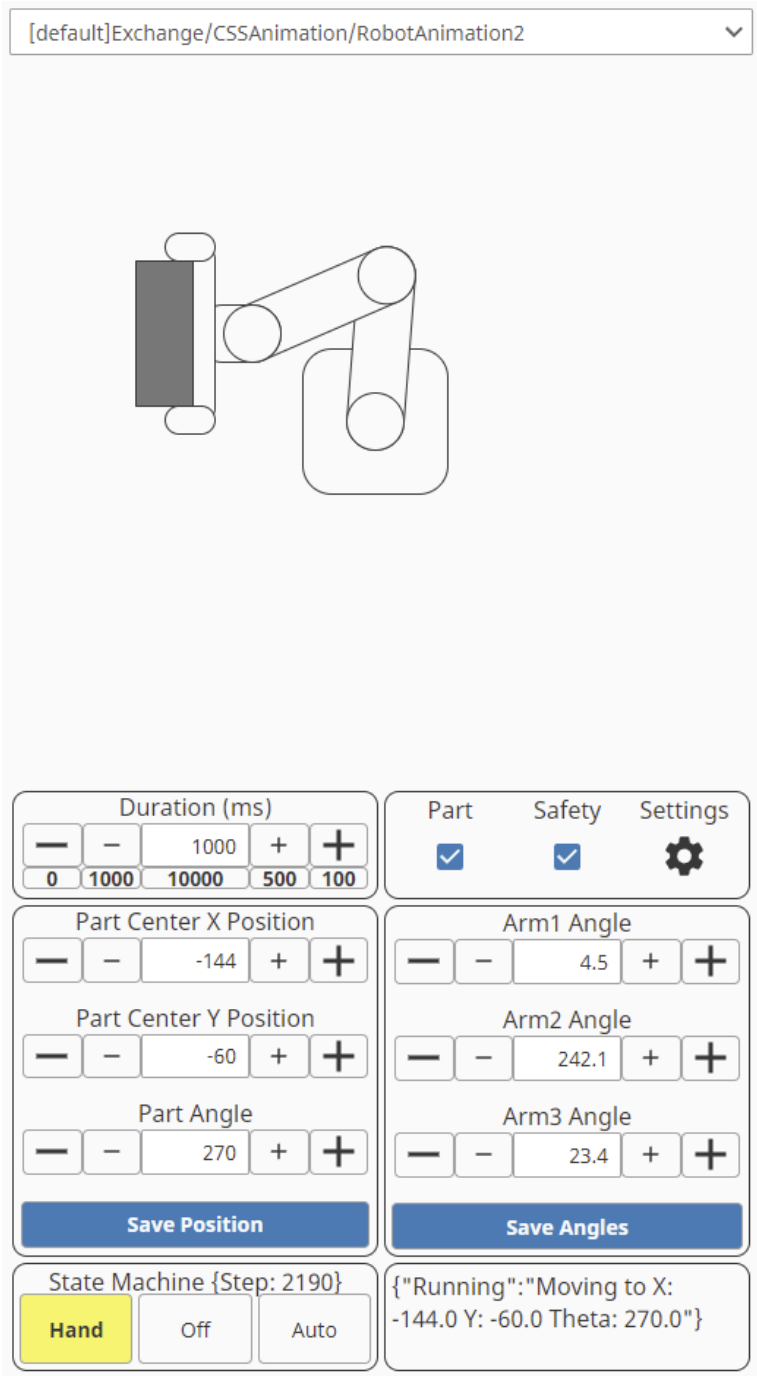


This screen is used to program the state machine and the robot animations.

All of the information is stored in tags. Feel free to modify these as much as desired, as they can be reverted to original by importing the tags again.



Animation



Robot Selection

Robot Animation

Duration - duration of animation
Part - display part or not
Safety - SafetyOK tag of UDT
Settings - See next page

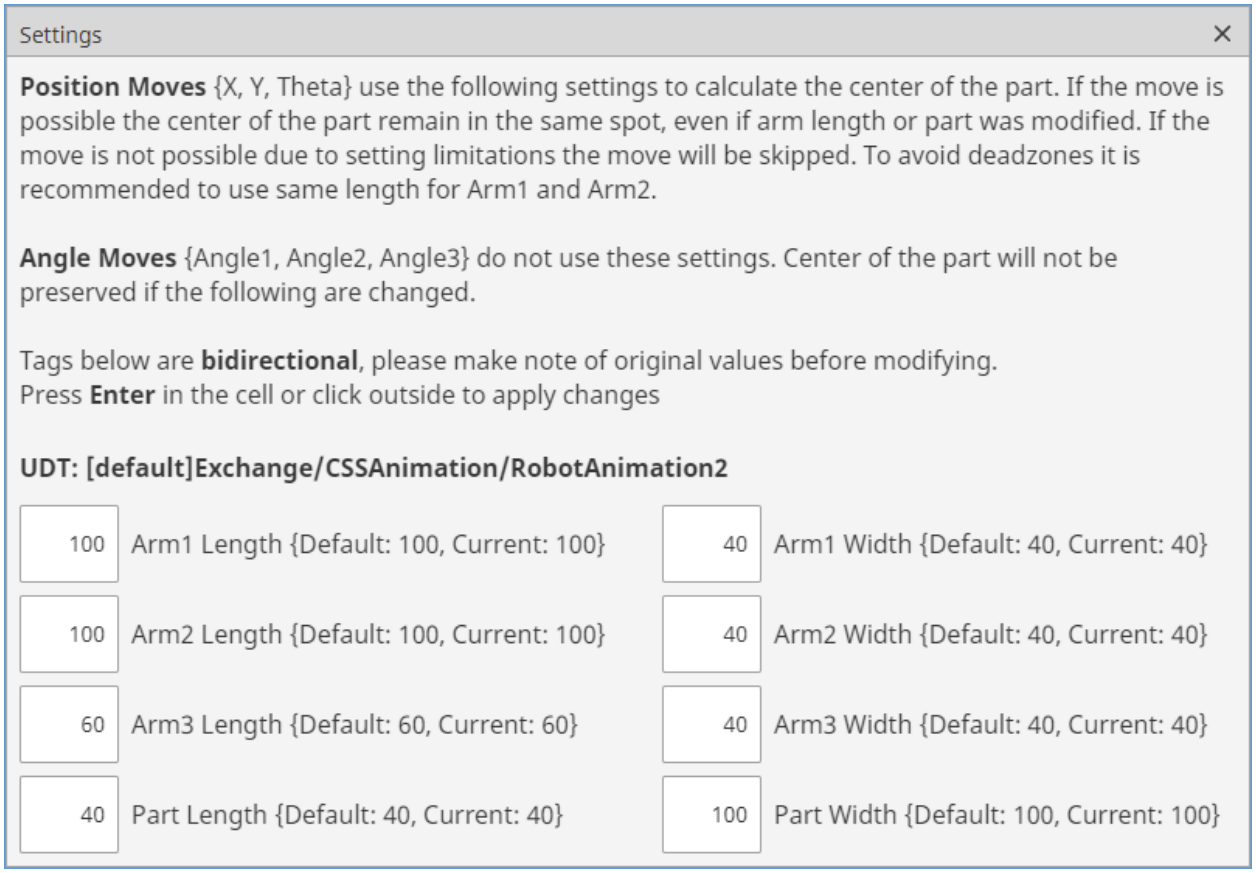
Save Position - writes current X, Y, Theta position to Tag Writes of the current step (Save needed)

Save Angles - writes current joint angles to Tag Writes of the current step (Save needed)

Left - **State Machine HOA**
Right - **Step Status**

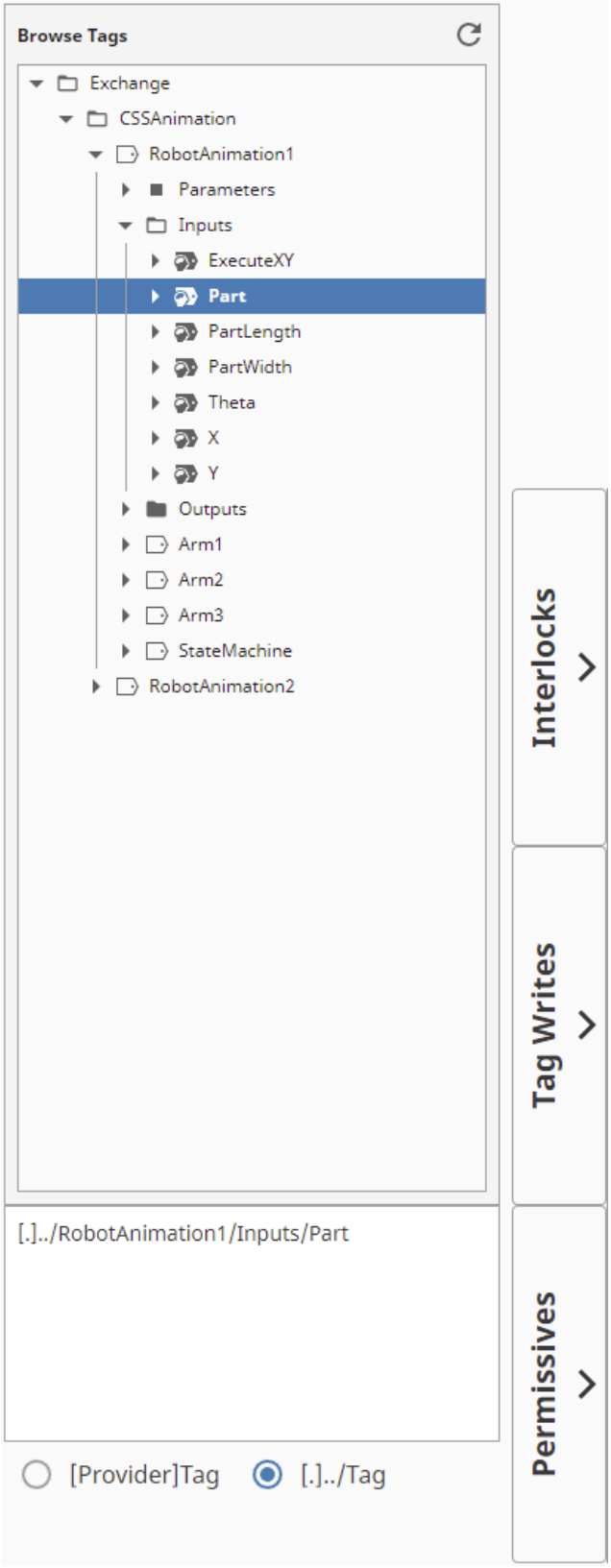
Settings and XY Motion

This feature can be used to change the dimensions of the robots. Please note that robot 1 uses joint moves so it will not preserve XY positions if altered. Robot 2 is using XY positions and can be modified easily. If the robot is unable to reach specified position the animation will be skipped.



Browse Tags

Tag browser allows for easy selection of tags to use in the state machine



Selected Tag(s)

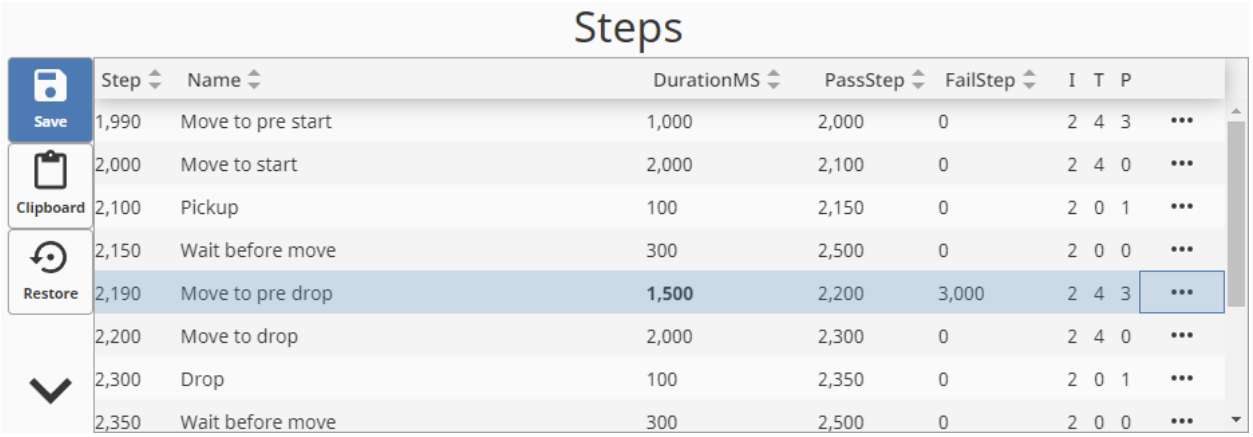
Static or dynamic Tag addressing

Interlocks - send currently selected tags to the interlock table. Values must then be assigned manually. (Save needed)

Tag Writes - send currently selected tags to the tag writes table. Values must then be assigned manually. (Save needed)

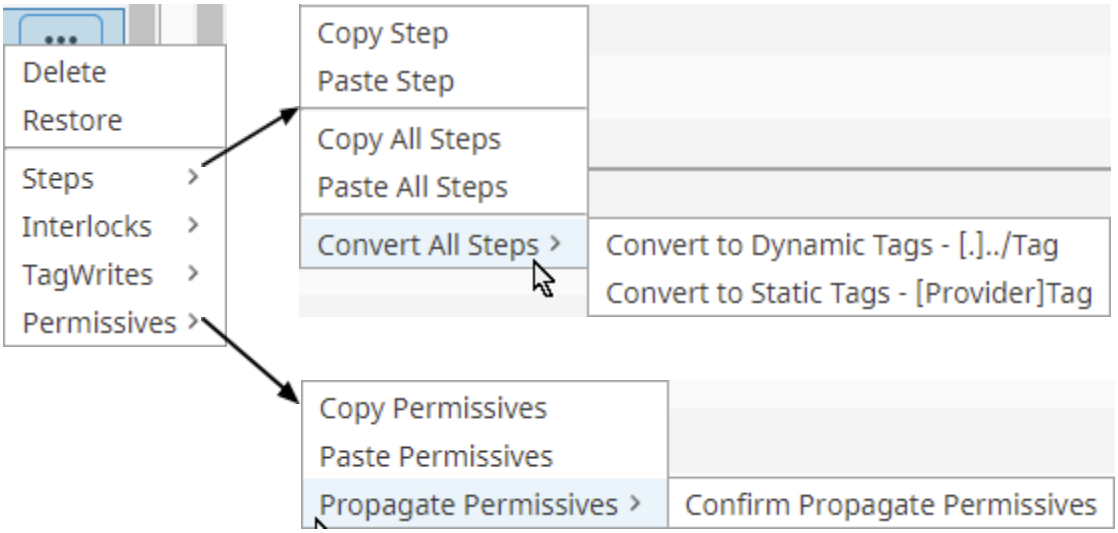
Permissives - send currently selected tags to the permissives table. Values must then be assigned manually. (Save needed)

Steps



- Left buttons
  - **Save** - will become enabled when there are any changes detected to the steps. Table elements that have been modified will become bolded as seen in the screenshot of step 2190 DurationMS.
  - **Clipboard** - used for copy and paste from step to step, or to backup data manually. The data can be modified directly in the clipboard or externally in a text editor, more details.
  - **Restore** - will revert any changes made to the steps
  - **V** - will expand steps to take up more vertical space
- Table
  - **Step** - number of the step
  - **Name** - name of the step
  - **DurationMS** - duration of the state machine step and animation in milliseconds
  - **PassStep** - step to which state machine will proceed if current step is successful
  - **FailStep** - step to which state machine will proceed if current step fails
  - **I** - count of interlocks in a step
  - **T** - count of tag writes in a step
  - **P** - count of permissives in a step
  - **…** - row actions, more details

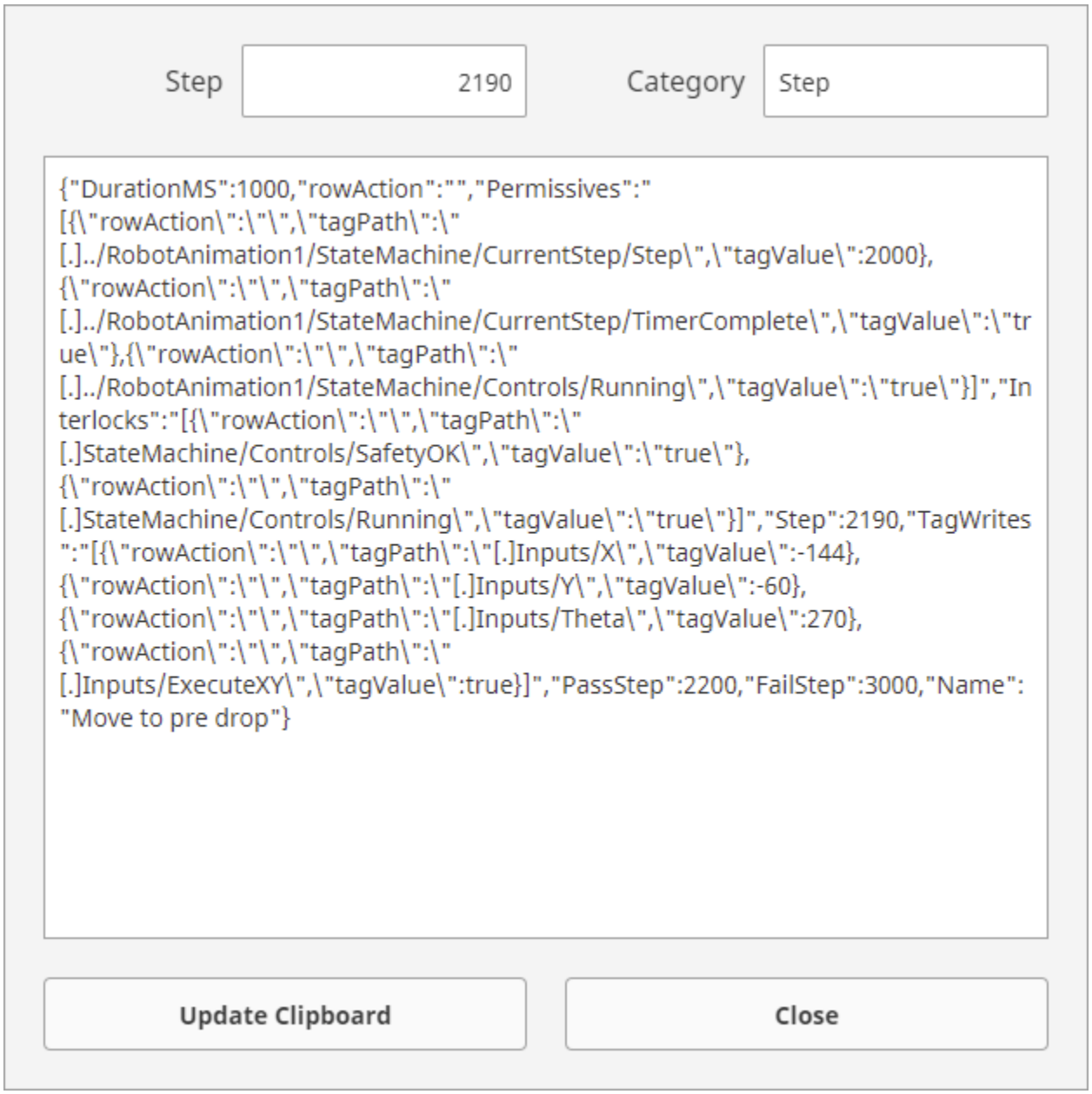
Row Actions



- Row actions - all actions require **Save** to take effect
  - **Delete** - deletes current step
  - **Restore** - only functional if step was deleted but not saved
  - **Steps**
    - **Copy Step** - sends current step to clipboard
    - **Paste Step** - overwrites step with data from clipboard
    - **Copy All Steps** - sends all steps to clipboard, useful for editing in external text editor and back up of one robot at a time
    - **Paste All Steps** - overwrites all steps with data from clipboard
    - **Convert All Steps**
      - **Convert to Dynamic Tags - [.]../Tag**
      - **Convert to Static Tags - [Provider]Tag**
  - **Interlocks**
    - **Copy Interlocks** - sends current step interlocks to clipboard
    - **Paste Interlocks** - overwrites interlocks with data from clipboard
    - **Propagate Interlocks** - applies interlocks to all other steps
  - **TagWrites**
    - **Copy TagWrites** - sends current step tag writes to clipboard
    - **Paste TagWrites** - overwrites tag writes with data from clipboard
    - **Propagate TagWrites** - applies tag writes to all other steps
  - **Permissives**
    - **Copy Permissives** - sends current step permissives to clipboard
    - **Paste Permissives** - overwrites permissives with data from clipboard
    - **Propagate Permissives** - applies permissives to all other steps

Clipboard

The clipboard is used for copy and paste from step to step, or to backup data manually. The data can be modified directly in this clipboard or externally in a text editor and then pasted back in. Note that this must be a valid JSON to import correctly. Press **Update Clipboard** to use newly modified values.



Interlocks

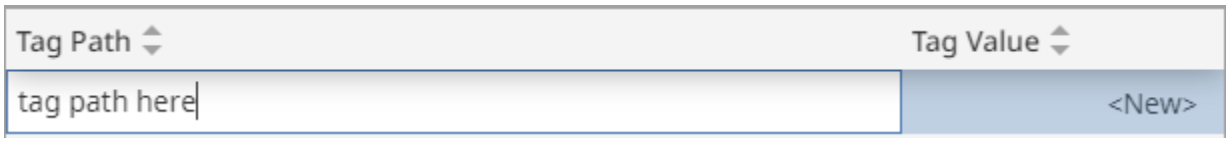
Interlocks must remain true for motion to continue. This allows for pausing and resuming of animation. Refer to the UDT page and StateMachine UDT for more information.

In this case the motion will be paused if SafetyOK or Running tags are false.

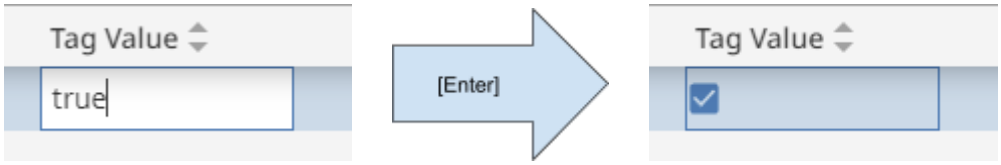
It is common practice to have the same interlocks in all steps.
The Propagate Interlocks tool can be used to send current step's interlocks to all other steps.

Manual Tag Value Assignment

New tags can be added by simply typing in the <New> row then pressing enter.
Or by pressing the **Interlocks >** button while tags are selected in the Tag Browser.



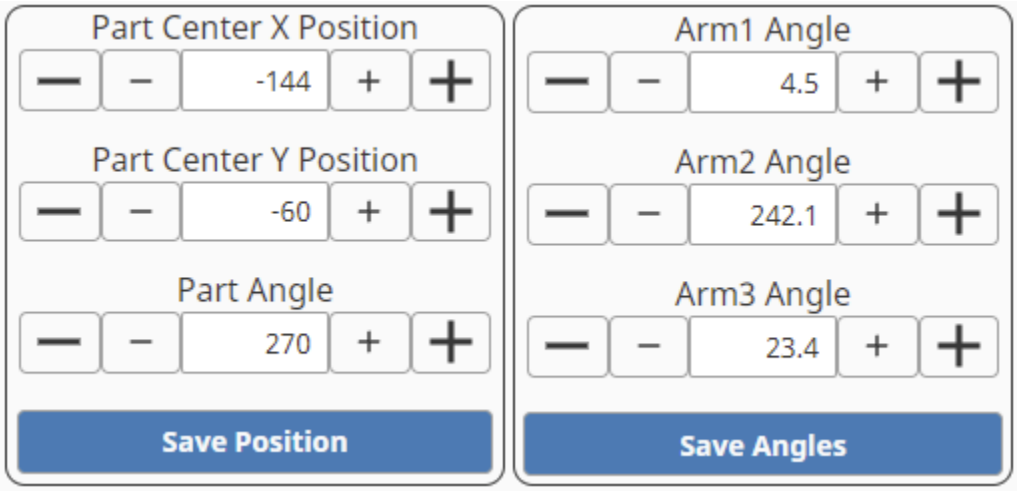
Note:
When typing tag path manually or using tag browser to send **Tag Path** to a table the **Tag Value** must be assigned manually. For boolean values type "true" or "false" then press enter, it will automatically convert it to a checkbox.



Tag Writes

Configured tag values will be written to the respective tag paths once the state machine enters this step and interlocks are satisfied.

For best animation programming experience it is recommended to use **Save Position** and **Save Angles** buttons on the screen.



Note:
Save Position and Save Angles also send Duration directly to the step, and Part to Tag Writes. If not desired it should be removed or modified manually before saving.

Permissives

Permissives

Tag Path	Tag Value
[.]../RobotAnimation1/StateMachine/CurrentStep/Step	2,000
[.]../RobotAnimation1/StateMachine/CurrentStep/TimerComplete	☑
[.]../RobotAnimation1/StateMachine/Controls/Running	☑
	<New>

Permissives must all be true in order for the state machine to move onto PassStep once the step timer is complete, otherwise it will proceed to FailStep. If FailStep is 0 the state machine will remain on this step indefinitely.

Examples:

Screenshot above is of permissives of step 2190 of RobotAnimation2
Screenshot below is of steps 2190 - 2300 of RobotAnimation2

Step	Name	DurationMS	PassStep	FailStep	I	T	P
2,190	Move to pre drop	1,000	2,200	3,000	2	4	3
2,200	Move to drop	2,000	2,300	0	2	4	0
2,300	Drop	100	2,350	0	2	0	1

For step 2190 the RobotAnimation2 will evaluate the state of RobotAnimation1. If RobotAnimation1 is on step 2000, that step is complete, and it is currently in running mode, the state machine of RobotAnimation2 will proceed to PassStep of 2200. If any of those are false it will proceed to FailStep of 3000.

For step 2200 there are no permissives, so the state machine will always proceed to PassStep.

For step 2300 there is one permissive and FailStep is 0, so the state machine will wait indefinitely on step 2300 until that permissive is met.